

AEDES II - Gabarito Comentado da Lista de Preparação

Este gabarito está separado dos enunciados para permitir tentativa prévia. Nas questões abertas, são apresentadas respostas-modelo; soluções logicamente equivalentes também são válidas.

Gabarito do Módulo 1

Questão 1

$\sum_{i=1..n} i = n(n+1)/2$, portanto $\Theta(n^2)$.

Questão 2

A função exata preserva constantes e termos menores; O limita superiormente, Ω inferiormente e Θ limita pelos dois lados. Para $4n^2+7n+12$, as três classes justas são $\Theta(n^2)$; também pertence a $O(n^2)$ e $\Omega(n^2)$.

Questão 3

- (a) Falsa: se o primeiro elemento for o procurado, o melhor caso é constante. (b) Falsa: uma ABB pode degenerar. (c) Falsa: em AVL/2-3-4/alvinegra, inserção é logarítmica. (d) Verdadeira no modelo usual: sem ordenação ou índice não há busca binária correta. (e) Falsa: todas as chaves podem cair na mesma lista, produzindo $O(n)$.

Questão 4

Sequencial não exige ordenação e tem pior caso $O(n)$. Binária exige ordenação e tem pior caso $O(\log n)$, mas inserir no vetor ordenado pode exigir deslocar $O(n)$ posições.

Questão 5

Início: inserção/remoção $\Theta(n)$ por deslocamentos. Fim: inserção e remoção $\Theta(1)$ se houver capacidade. Posição: $\Theta(n)$ no pior caso. Localizar por valor pode acrescentar $\Theta(n)$.

Questão 6

Vazia quando `primeiro==ultimo`; cheia quando `(ultimo+1)%array.length==primeiro`. A posição extra distingue cheio de vazio. Inserção avança `ultimo`; remoção avança `primeiro`.

Questão 7

Desfazer: pilha. Atendimento: fila. Parênteses: pilha. Posições arbitrárias: lista. Largura de árvore: fila.

Questão 8

Sequenciais têm boa localidade e acesso por índice, mas capacidade fixa e deslocamentos. Flexíveis crescem dinamicamente e evitam deslocamentos, mas usam ponteiros, alocação e pior localidade.

Questão 9

Lista simples exige encontrar o anterior para religar $\text{ant.prox} = \text{alvo.prox}$; dupla altera alvo.ant.prox e alvo.prox.ant . Com referência direta, a dupla remove em $O(1)$; a simples pode precisar encontrar o anterior.

Questão 10

Partir da primeira célula de dados e, a cada passo, avançar inf e dir. As cabeças são sentinelas e não representam valores da matriz.

Questão 11

Bubble e Insertion são estáveis nas versões usuais e in-place. Selection não é estável na versão clássica. Merge é estável, mas usa memória auxiliar $O(n)$. Quick e Heap não são estáveis em geral. Counting e Radix podem ser estáveis se implementados com acumulação/ordenação estável.

Questão 12

Bubble otimizado e Insertion: melhor $\Theta(n)$ em vetor ordenado; pior $\Theta(n^2)$ em ordem inversa. Merge: $\Theta(n \log n)$ em todos os casos usuais; movimentos também $\Theta(n \log n)$.

Questão 13

Selection sempre faz $n(n-1)/2$ comparações. Quick degrada com partições 0 e $n-1$. Heap mantém $\Theta(n \log n)$. Counting depende de k pequeno ($\Theta(n+k)$); Radix depende da quantidade de dígitos e da base ($\Theta(d(n+b))$).

Questão 14

Quase ordenado pequeno: Insertion. Estabilidade/grande volume: Merge. Inteiros em faixa pequena: Counting. Pouca memória: Heap ou Insertion em pequeno volume. Pivô inicial e entrada ordenada: evitar Quick sem estratégia melhor de pivô.

Questão 15

Todos os valores da esquerda devem ser menores que o nó e todos os da direita maiores, em todo o domínio da subárvore. Balanceada: $O(\log n)$; degenerada: $O(n)$.

Questão 16

Folha: retorna nulo. Um filho: substitui pelo filho. Dois filhos: troca com predecessor (maior da esquerda) ou sucessor. maiorEsq copia o maior valor da subárvore esquerda e remove esse nó, mantendo os intervalos.

Questão 17

Todos os nós: soma/contagem. Um ramo: pesquisa ABB. Baixo para cima: altura ou validação AVL, retornando altura e validade.

Questão 18

FB -2 com filho esquerdo não positivo: direita; FB -2 com filho esquerdo positivo: esquerda no filho e direita; simétrico para +2. Atualizam-se primeiro os nós que descem e depois a nova raiz.

Questão 19

2-nó vira um nó binário branco; 3-nó é representado por ligação colorida; 4-nó por nó branco com dois filhos coloridos. Na descida, divide antes de entrar; na ascensão, divide depois do overflow e pode propagar.

Questão 20

A raiz é branca, não há dois nós coloridos consecutivos e todos os caminhos até nulos têm mesma altura branca. Recoloração corresponde à fragmentação de 4-nó; rotação corrige ligações coloridas consecutivas.

Questão 21

Reserva é simples, mas limitada; rehash oferece poucas alternativas e sofre agrupamento; encadeamento aceita várias colisões, mas usa memória de nós. Pior caso de todas pode ser linear, dependendo do esquema/carga.

Questão 22

Hash é boa para exata, ruim para prefixos. TRIE percorre caracteres e encontra prefixos naturalmente. PATRICIA comprime caminhos. folha distingue palavra completa de simples prefixo.

Questão 23

Custo $O(h + 1 + k)$ em uma hash de acesso constante médio, ou $O(h+k)$. No pior caso, h e k podem ser lineares. A notação deve manter os custos de cada camada.

Questão 24

- (a) V; (b) F; (c) F; (d) F, depende de dígitos/base; (e) V; (f) V; (g) F; (h) V, salvo atualizações de extremos/sentinelas necessárias, ainda constantes.

Gabarito do Módulo 2

Questão 25

Estados: [2,4,7,9,15]; depois [2,4,7,8,9,15]; remover(4) remove 9 -> [2,4,7,8,15]; removerFim remove 15 -> [2,4,7,8].

Questão 26

Após 10,20,30,40: p=0,u=4. Após duas remoções: p=2,u=4. Inserir 50 -> índice4, 60->5, 70->0, então u=1. Remover tira 30 (p=3). Inserir 80 no índice1 (u=2). Ordem lógica final: 40,50,60,70,80.

Questão 27

Pilha remove 5 e depois 2; final topo 8 sobre 3. Fila remove 3 e depois 8; final ordem 5,2.

Questão 28

Depois de 5,9,12: cabeça->5->9->12 e ultimo=12. Inserir 7 pos1: 5->7->9->12. Remover início: 7->9->12. Remover pos1: 7->12.

Questão 29

Remover 8 liga 4.prox=15 e 15.ant=4. Inserir 10 entre 15 e16: 15<->10<->16. Remover fim faz ultimo=10, 10.prox=null.

Questão 30

Caminho: célula(0,0)->inf.dir->inf.dir. A lista resultado contém, na ordem, todos os elementos da lista de 1, depois 5, depois 9; cada elemento é copiado para nova CelulaDupla.

Questão 31

Passagens: [2,5,1,4,7], [2,1,4,5,7], [1,2,4,5,7]; a última passagem pode parar sem trocas. A última troca reduz o limite.

Questão 32

Estados esperados: 4,3 -> [3,4,...]; com 8 [3,4,8]; 6 [3,4,6,8]; 1 [1,3,4,6,8]; 2 [1,3,2,4,6,8]; 9 [1,3,9,2,4,6,8]; 0 fica no grupo par ordenado; 5 e 7 entram no grupo ímpar. Final [1,3,5,7,9,0,2,4,6,8].

Questão 33

Divisões: [38,27,43,3] e [9,82,10]; depois unitários. Intercalações principais: [27,38], [3,43], [3,27,38,43], [9,82], [9,10,82], final [3,9,10,27,38,43,82].

Questão 34

Com pivô 3, os ponteiros avançam e realizam trocas até cruzar. Uma partição possível do código clássico resulta [2,1,3,8,4,9,5] antes das chamadas seguintes; pequenas diferenças dependem do exato laço, mas os elementos à esquerda/direita do pivô ficam particionados.

Questão 35

Árvore: raiz15; esquerda8 com filhos4 e12; 12 com 10 e14; direita22 com 18 e30. Central: 4 8 10 12 14 15 18 22 30. Pré: 15 8 4 12 10 14 22 18 30. Pós: 4 10 14 12 8 18 30 22 15.

Questão 36

Remover10 elimina folha. Remover12, então com apenas filho14, substitui por14. Remover15 usa maior da esquerda: 14 substitui a raiz e o antigo14 é retirado da subárvore esquerda.

Questão 37

A ABB produz 1,3,5,7,8,9,10 no caminhamento central. São 7 inserções e 7 visitas para preencher 7 posições.

Questão 38

- (a) rotação direita em30 ->20 raiz. (b) esquerda em10 ->20. (c) esquerda em10 e direita em30 ->20. (d) direita em30 e esquerda em10 ->20.

Questão 39

Árvore final: raiz12; esquerda8, com 4(2,6) e10; direita18, com14(dir16) e20(dir22). Níveis: raiz4, 8 e18 nível3, 4/14/20 nível2, folhas nível1. FB da raiz0; 8=-1;18=0;14=1;20=1.

Questão 40

Árvore final 2-3-4: raiz [8 | 12 | 18]; filhos [2 | 4 | 6], [10], [14 | 16], [20 | 22].

Questão 41

A árvore final é a mesma: raiz [8 | 12 | 18] e quatro filhos [2 | 4 | 6], [10], [14 | 16], [20 | 22]; as diferenças estão no momento em que o overflow é tratado.

Questão 42

Uma representação final compatível com o algoritmo do material: 12(B); esquerda 8(P), com 4(B) filhos2(P),6(P), e 10(B); direita18(P), com14(B) dir16(P), e20(B) dir22(P). Aqui B=branco e P=colorido/preto conforme convenção declarada.

Questão 43

Reserva: 14 na pos0; 21 colide e vai reserva; 8 pos1;15 colide;22 colide;3 pos3;10 colide/overflow conforme capacidade. Rehash usa a segunda posição $(x+1)\%7$, podendo falhar após duas

ocupações. Encadeamento: índice0 lista14->21; índice1 8->15->22; índice3 3->10. Pesquisa22 percorre lista1;17 procura índice3 e falha após a lista.

Questão 44

TRIE compartilha BRA entre BRASIL/BRASA/BRAVO e CHI entre CHILE/CHINA; finais em L,A,O,E,A. PATRICIA comprime segmentos sem bifurcação, por exemplo BRA e CHI. Na doidona, busca BRASA: árvore de letra B -> hash do tamanho5 -> lista/ABB do bucket -> comparação da palavra.

Gabarito do Módulo 3

Questão 45

```
public static int contarMaiusculas(String s) {  
    return contarMaiusculas(s, 0);  
}  
  
private static int contarMaiusculas(String s, int i) {  
    int resp = 0;  
    if (i < s.length()) {  
        resp = ((s.charAt(i) >= 'A' && s.charAt(i) <= 'Z') ? 1 : 0)  
            + contarMaiusculas(s, i + 1);  
    }  
    return resp;  
}
```

Tempo $O(|s|)$ e pilha $O(|s|)$.

Questão 46

Há duas multiplicações por par (i,j). Logo $2\sum_{i=1..n} i = n(n+1)$. A indução usa base $n=1$ e passo $F(n+1)=F(n)+2(n+1)$. Ordem $\Theta(n^2)$.

Questão 47

```
public void inserirOrdenado(int x) throws Exception {  
    if (n >= array.length) {  
        throw new Exception("Erro ao inserir!");  
    }  
    int i;  
    for (i = n - 1; i >= 0 && array[i] > x; i--) {  
        array[i + 1] = array[i];  
    }  
    array[i + 1] = x;  
}
```

```
    n++;  
}
```

Pior caso $O(n)$; melhor $O(1)$ quando entra no fim.

Questão 48

```
public int removerTodos(int x) {  
    int escrita = 0;  
    int removidos = 0;  
    for (int leitura = 0; leitura < n; leitura++) {  
        if (array[leitura] == x) {  
            removidos++;  
        } else {  
            array[escrita++] = array[leitura];  
        }  
    }  
    n = escrita;  
    return removidos;  
}
```

$O(n)$ tempo e $O(1)$ espaço.

Questão 49

```
public int somaRec() {  
    return somaRec(primeiro);  
}  
  
private int somaRec(int i) {  
    int resp = 0;  
    if (i != ultimo) {  
        resp = array[i] + somaRec((i + 1) % array.length);  
    }  
    return resp;  
}
```

Questão 50

```
public static boolean balanceada(String s) throws Exception {  
    PilhaChar p = new PilhaChar();  
    boolean resp = true;  
    for (int i = 0; i < s.length() && resp; i++) {  
        char c = s.charAt(i);  
        if (c == '(' || c == '[' || c == '{') {  
            p.inserir(c);  
        } else if (c == ')' || c == ']' || c == '}') {
```

```

        if (p.vazia()) {
            resp = false;
        } else {
            char a = p.remover();
            resp = (a == '(' && c == ')') ||
                (a == '[' && c == ']') ||
                (a == '{' && c == '}');
        }
    }
}
return resp && p.vazia();
}

```

Tempo $O(n)$ e espaço $O(n)$.

Questão 51

```

public int getMax() throws Exception {
    if (topo == null) {
        throw new Exception("Pilha vazia!");
    }
    return getMax(topo);
}

private int getMax(Celula i) {
    int resp = i.elemento;
    if (i.prox != null) {
        int maxResto = getMax(i.prox);
        resp = (resp > maxResto) ? resp : maxResto;
    }
    return resp;
}

```

Questão 52

```

public static void inverter(Fila fila) throws Exception {
    Pilha pilha = new Pilha();
    while (!fila.vazia()) {
        pilha.inserir(fila.remover());
    }
    while (!pilha.vazia()) {
        fila.inserir(pilha.remover());
    }
}

```


Questão 53

```
public void inserirOrdenado(int x) {
    Celula ant = primeiro;
    Celula i = primeiro.prox;
    while (i != null && i.elemento < x) {
        ant = i;
        i = i.prox;
    }
    Celula nova = new Celula(x);
    nova.prox = i;
    ant.prox = nova;
    if (i == null) {
        ultimo = nova;
    }
}
```

Questão 54

```
public int removerValor(int x) throws Exception {
    Celula ant = primeiro;
    Celula i = primeiro.prox;
    while (i != null && i.elemento != x) {
        ant = i;
        i = i.prox;
    }
    if (i == null) {
        throw new Exception("Elemento inexistente!");
    }
    ant.prox = i.prox;
    if (i == ultimo) {
        ultimo = ant;
    }
    i.prox = null;
    return i.elemento;
}
```

Questão 55

```
public boolean isPalindromo() {
    CelulaDupla esq = primeiro.prox;
    CelulaDupla dir = ultimo;
    boolean resp = true;
    while (esq != null && dir != primeiro && esq != dir && esq.ant != dir && resp) {
        resp = (esq.elemento == dir.elemento);
        esq = esq.prox;
    }
}
```

```

        dir = dir.ant;
    }
    return resp;
}

```

$O(n)$ tempo e $O(1)$ espaço.

Questão 56

```

public CelulaDupla diagUnificada() {
    CelulaDupla cabeca = new CelulaDupla();
    CelulaDupla ultimo = cabeca;
    CelulaMatriz m = inicio;

    while (m != null) {
        for (Celula i = m.inicio.prox; i != null; i = i.prox) {
            CelulaDupla nova = new CelulaDupla();
            nova.elemento = i.elemento;
            nova.ant = ultimo;
            ultimo.prox = nova;
            ultimo = nova;
        }
        m = (m.inf != null) ? m.inf.dir : null;
    }
    return cabeca.prox;
}

```

Na prática, deve-se ajustar o ponto inicial se a matriz possuir sentinelas de linha/coluna. Custo proporcional à diagonal mais elementos copiados.

Questão 57

```

public void sort() {
    int limite = n - 1;
    while (limite > 0) {
        int ultimaTroca = 0;
        for (int j = 0; j < limite; j++) {
            comparacoes++;
            if (array[j] > array[j + 1]) {
                swap(j, j + 1);
                movimentacoes += 3;
                ultimaTroca = j;
            }
        }
        limite = ultimaTroca;
    }
}

```

```
}  
}
```

Questão 58

```
private boolean vemAntes(int a, int b) {  
    boolean imparA = Math.abs(a % 2) == 1;  
    boolean imparB = Math.abs(b % 2) == 1;  
    if (imparA != imparB) {  
        return imparA;  
    }  
    return a < b;  
}  
  
public void sort() {  
    for (int i = 1; i < n; i++) {  
        int tmp = array[i];  
        int j = i - 1;  
        while (j >= 0 && vemAntes(tmp, array[j])) {  
            array[j + 1] = array[j--];  
        }  
        array[j + 1] = tmp;  
    }  
}
```

Questão 59

```
private void intercalar(int esq, int meio, int dir) {  
    int[] tmp = new int[dir - esq + 1];  
    int i = esq, j = meio + 1, k = 0;  
    while (i <= meio && j <= dir) {  
        if (array[i] >= array[j]) tmp[k++] = array[i++];  
        else tmp[k++] = array[j++];  
    }  
    while (i <= meio) tmp[k++] = array[i++];  
    while (j <= dir) tmp[k++] = array[j++];  
    for (k = 0; k < tmp.length; k++) array[esq + k] = tmp[k];  
}
```

Questão 60

```
private boolean vemAntes(Aluno a, Aluno b) {  
    if (a.nota != b.nota) return a.nota > b.nota;  
    return a.nome.compareTo(b.nome) <= 0;  
}
```

```

private void intercalar(int esq, int meio, int dir) {
    Aluno[] tmp = new Aluno[dir - esq + 1];
    int i = esq, j = meio + 1, k = 0;
    while (i <= meio && j <= dir) {
        if (vemAntes(array[i], array[j])) tmp[k++] = array[i++];
        else tmp[k++] = array[j++];
    }
    while (i <= meio) tmp[k++] = array[i++];
    while (j <= dir) tmp[k++] = array[j++];
    for (k = 0; k < tmp.length; k++) array[esq + k] = tmp[k];
}

```

Questão 61

Erros: tamanho deveria ser dir-esq+1; a segunda metade começa em meio+1; é preciso testar esgotamento dos lados; e copiar tmp de volta.

```

private void intercalar(int esq, int meio, int dir) {
    int[] tmp = new int[dir - esq + 1];
    int i = esq, j = meio + 1, k = 0;
    while (i <= meio && j <= dir) {
        if (array[i] <= array[j]) tmp[k++] = array[i++];
        else tmp[k++] = array[j++];
    }
    while (i <= meio) tmp[k++] = array[i++];
    while (j <= dir) tmp[k++] = array[j++];
    for (k = 0; k < tmp.length; k++) array[esq + k] = tmp[k];
}

```

Questão 62

Se todos forem pares, cada elemento executa $\lceil \log_2 n \rceil + 1$ multiplicações: $\Theta(n \log n)$. Se todos forem ímpares, o total é $\sum_{i=0..n-1} i = n(n-1)/2$: $\Theta(n^2)$. Esses são melhor/pior para número de multiplicações quando n é grande.

Questão 63

```

public boolean pesquisar(int x) {
    comparacoes = 0;
    return pesquisar(x, raiz);
}

private boolean pesquisar(int x, No i) {
    boolean resp;
    if (i == null) {
        resp = false;
    }
}

```

```

    } else {
        comparacoes++;
        if (x == i.elemento) resp = true;
        else if (x < i.elemento) resp = pesquisar(x, i.esq);
        else resp = pesquisar(x, i.dir);
    }
    return resp;
}

```

Questão 64

```

public int contarFolhas() { return contarFolhas(raiz); }
private int contarFolhas(No i) {
    int resp = 0;
    if (i != null) {
        if (i.esq == null && i.dir == null) resp = 1;
        else resp = contarFolhas(i.esq) + contarFolhas(i.dir);
    }
    return resp;
}

public int contarInternos() { return contarInternos(raiz); }
private int contarInternos(No i) {
    int resp = 0;
    if (i != null && (i.esq != null || i.dir != null)) {
        resp = 1 + contarInternos(i.esq) + contarInternos(i.dir);
    }
    return resp;
}

```

Questão 65

```

public boolean isMax(double valor) {
    int n = quantidade(raiz);
    return n == 0 || altura(raiz) <= valor * (Math.log(n) / Math.log(2));
}

private int quantidade(No i) {
    return (i == null) ? 0 : 1 + quantidade(i.esq) + quantidade(i.dir);
}

private int altura(No i) {
    return (i == null) ? -1 : 1 + Math.max(altura(i.esq), altura(i.dir));
}

```

Questão 66

```
public boolean isABB() {
    return isABB(raiz, null, null);
}

private boolean isABB(No i, Integer min, Integer max) {
    boolean resp = true;
    if (i != null) {
        if ((min != null && i.elemento <= min) || (max != null && i.elemento >= max)) {
            resp = false;
        } else {
            resp = isABB(i.esq, min, i.elemento) && isABB(i.dir, i.elemento, max);
        }
    }
    return resp;
}
```

Questão 67

```
public ArvoreBinaria obterSubconjuntoMenores(int x) throws Exception {
    if (!pesquisar(x)) throw new Exception("Elemento inexistente!");
    ArvoreBinaria resp = new ArvoreBinaria();
    copiarMenores(raiz, x, resp);
    return resp;
}

private void copiarMenores(No i, int x, ArvoreBinaria destino) throws Exception {
    if (i != null) {
        if (i.elemento <= x) {
            destino.inserir(i.elemento);
            copiarMenores(i.esq, x, destino);
            copiarMenores(i.dir, x, destino);
        } else {
            copiarMenores(i.esq, x, destino);
        }
    }
}
```

Questão 68

```
public boolean pesquisar(String s) {
    return pesquisar(raiz, s);
}

private boolean pesquisar(No i, String s) {
    boolean resp;
    char c = s.charAt(0);
    if (i == null) resp = false;
```

```

else if (c < i.letra) resp = pesquisar(i.esq, s);
else if (c > i.letra) resp = pesquisar(i.dir, s);
else resp = pesquisar2(i.raiz2, s);
return resp;
}
private boolean pesquisar2(No2 i, String s) {
    if (i == null) return false;
    if (s.equals(i.palavra)) return true;
    return (s.compareTo(i.palavra) < 0) ? pesquisar2(i.esq, s) : pesquisar2(i.dir, s);
}
public int contarComTamanho(int tam) { return contar1(raiz, tam); }
private int contar1(No i, int tam) {
    return (i == null) ? 0 : contar2(i.raiz2, tam) + contar1(i.esq, tam) +
    contar1(i.dir, tam);
}
private int contar2(No2 i, int tam) {
    return (i == null) ? 0 : ((i.palavra.length() == tam) ? 1 : 0) + contar2(i.esq, tam)
    + contar2(i.dir, tam);
}

```

Questão 69

```

public boolean pesquisar(int cpf) { return pesquisarCpf(raiz, cpf); }
private boolean pesquisarCpf(No i, int cpf) {
    if (i == null) return false;
    return pesquisarLista(i.primeiro.prox, cpf) || pesquisarCpf(i.esq, cpf) ||
    pesquisarCpf(i.dir, cpf);
}
private boolean pesquisarLista(Celula i, int cpf) {
    return i != null && (i.contato.cpf == cpf || pesquisarLista(i.prox, cpf));
}
public boolean pesquisarNome(String nome) { return pesquisarNome(raiz, nome); }
private boolean pesquisarNome(No i, String nome) {
    if (i == null) return false;
    char c = nome.charAt(0);
    if (c < i.letra) return pesquisarNome(i.esq, nome);
    if (c > i.letra) return pesquisarNome(i.dir, nome);
    for (Celula j = i.primeiro.prox; j != null; j = j.prox)
        if (j.contato.nome.equals(nome)) return true;
    return false;
}

```

Questão 70

```
public void setNivel() {
    nivel = 1 + Math.max(getNivel(esq), getNivel(dir));
}
public static int getNivel(No no) {
    return (no == null) ? 0 : no.nivel;
}
public int getFatorBalanceamento() {
    return getNivel(dir) - getNivel(esq);
}
```

Questão 71

```
private static class Info { boolean ok; int nivel; int min, max; boolean vazio; }
public boolean isAVL() { return validar(raiz).ok; }
private Info validar(No i) {
    Info r = new Info();
    if (i == null) { r.ok=true; r.nivel=0; r.vazio=true; return r; }
    Info e=validar(i.esq), d=validar(i.dir);
    int nivel=1+Math.max(e.nivel,d.nivel);
    boolean ordem=(e.vazio || e.max<i.elemento)&&(d.vazio || d.min>i.elemento);
    r.ok=e.ok&&d.ok&&ordem&&Math.abs(d.nivel-e.nivel)<=1&&i.nivel==nivel;
    r.nivel=nivel; r.vazio=false; r.min=e.vazio?i.elemento:e.min; r.max=d.vazio?
i.elemento:d.max;
    return r;
}
```

Questão 72

```
private No balancear(No no) throws Exception {
    if (no != null) {
        int fator = No.getNivel(no.dir) - No.getNivel(no.esq);
        if (Math.abs(fator) <= 1) {
            no.setNivel();
        } else if (fator == 2) {
            int fd = No.getNivel(no.dir.dir) - No.getNivel(no.dir.esq);
            if (fd < 0) no.dir = rotacionarDir(no.dir);
            no = rotacionarEsq(no);
        } else if (fator == -2) {
            int fe = No.getNivel(no.esq.dir) - No.getNivel(no.esq.esq);
            if (fe > 0) no.esq = rotacionarEsq(no.esq);
            no = rotacionarDir(no);
        } else throw new Exception("Fator invalido: " + fator);
    }
}
```



```
    return no;
}
```

Questão 73

```
private No remover(int x, No i) throws Exception {
    if (i == null) throw new Exception("Erro ao remover!");
    if (x < i.elemento) i.esq = remover(x, i.esq);
    else if (x > i.elemento) i.dir = remover(x, i.dir);
    else if (i.dir == null) i = i.esq;
    else if (i.esq == null) i = i.dir;
    else i.esq = maiorEsq(i, i.esq);
    return balancear(i);
}
```

Cada nó do caminho pode ficar desbalanceado; por isso o balanceamento ocorre em cada retorno recursivo.

Questão 74

Uma solução $O(n)$ retorna um objeto com validade, menor, maior, nível da primeira folha e quantidade de chaves. Para cada nó: validar 1.3 chaves, ordem interna, intervalos recebidos, exatamente $qtdChaves+1$ filhos se não for folha e o mesmo nível de folha em todos os ramos.

Questão 75

```
public boolean mesmaQuantidadeBrancos() {
    return raiz == null || contarBrancos(raiz.esq) == contarBrancos(raiz.dir);
}
private int contarBrancos(NoAN i) {
    return (i == null) ? 0 : ((i.cor) ? 1 : 0) + contarBrancos(i.esq) + contarBrancos(i.dir);
}
```

Questão 76

```
private static class Info { boolean ok; int brancos; }
public boolean equilibradaBrancaGlobal() { return verificar(raiz).ok; }
private Info verificar(NoAN i) {
    Info r = new Info();
    if (i == null) { r.ok=true; r.brancos=0; return r; }
    Info e=verificar(i.esq), d=verificar(i.dir);
    r.ok=e.ok && d.ok && e.brancos==d.brancos;
    r.brancos=e.brancos+d.brancos+((i.cor)?1:0);
    return r;
}
```

Questão 77

```
public boolean verificaArvore() {
    return raiz == null || (!raiz.cor && alturaBranca(raiz) >= 0);
}

private int alturaBranca(NoAN i) {
    if (i == null) return 0;
    if (i.cor && ((i.esq != null && i.esq.cor) || (i.dir != null && i.dir.cor))) return -1;
    int e=alturaBranca(i.esq), d=alturaBranca(i.dir);
    if (e < 0 || d < 0 || e != d) return -1;
    return e + ((!i.cor) ? 1 : 0);
}
```

Questão 78

Pesquisa: testar principal e depois reserva. Remoção principal: guardar removido, procurar na reserva o primeiro y com $h(y) == pos$, movê-lo para principal e deslocar a reserva à esquerda; remoção na reserva apenas compacta. Todos os passos na reserva são $O(m^2)$ no pior caso.

Questão 79

```
static final int NULO=-1, REMOVIDO=-2;

public boolean inserir(int x) {
    int p=h(x); if (tabela[p]==NULO || tabela[p]==REMOVIDO) { tabela[p]=x; return true; }
    p=reh(x); if (tabela[p]==NULO || tabela[p]==REMOVIDO) { tabela[p]=x; return true; }
    return false;
}

public boolean pesquisar(int x) {
    return tabela[h(x)]==x || tabela[reh(x)]==x;
}

public boolean remover(int x) {
    int p=h(x); if (tabela[p]==x) { tabela[p]=REMOVIDO; return true; }
    p=reh(x); if (tabela[p]==x) { tabela[p]=REMOVIDO; return true; }
    return false;
}
```

Questão 80

Cada bucket usa a lista do material. maiorLista percorre todas as m listas e conta seus nós, retornando o maior. Tempo $O(m+n)$; pesquisa média $O(1+\alpha)$, pior $O(n)$.

Questão 81

```
public boolean pesquisar(String s) { return pesquisar(s, raiz, 0); }
private boolean pesquisar(String s, No no, int i) {
    No f=no.prox[No.hash(s.charAt(i))];
```

```

    if (f==null) return false;
    if (i==s.length()-1) return f.folha;
    return pesquisar(s,f,i+1);
}
public boolean existePrefixo(String p) {
    No no=raiz;
    for (int i=0;i<p.length();i++) {
        no=no.prox[No.hash(p.charAt(i))];
        if (no==null) return false;
    }
    return temPalavra(no);
}
private boolean temPalavra(No no) {
    if (no.folha) return true;
    for (No f:no.prox) if (f!=null && temPalavra(f)) return true;
    return false;
}

```

Questão 82

```

public boolean existePrefixoSufixo(String p, String suf) {
    No no=raiz;
    for (int i=0;i<p.length();i++) {
        no=no.prox[No.hash(p.charAt(i))];
        if (no==null) return false;
    }
    return buscarSufixo(no, p, suf);
}
private boolean buscarSufixo(No no, String atual, String suf) {
    if (no.folha && atual.endsWith(suf)) return true;
    for (No f:no.prox) if (f!=null && buscarSufixo(f, atual+f.elemento, suf)) return true;
    return false;
}

```

Questão 83

Pesquisa compara o próximo segmento caractere a caractere; só retorna true se consumir toda a palavra, todo o segmento e chegar a nó folha. contarAs soma as letras A do segmento de cada nó e visita recursivamente cada filho uma única vez. Custo linear no total de caracteres armazenados nos rótulos visitados.

Questão 84

```

public boolean pesquisar(int elemento) {
    int p=hashT1(elemento);
    if (t1[p]==elemento) return true;
}

```

```

if (t1[p]==NULO) return false;
int destino=hashT2(elemento);
if (destino==0) {
    p=hashT3(elemento);
    if (t3[p]==elemento) return true;
    p=rehashT3(elemento);
    return t3[p]==elemento || pesquisarArvore(raizArvoreT3,elemento);
}
if (destino==1) {
    for (Celula i=primeiroLista.prox;i!=null;i=i.prox)
        if (i.elemento==elemento) return true;
    return false;
}
return pesquisarArvore(raizArvoreT2,elemento);
}
private boolean pesquisarArvore(No i,int x) {
    if (i==null) return false;
    if (x==i.elemento) return true;
    return (x<i.elemento)?pesquisarArvore(i.esq,x):pesquisarArvore(i.dir,x);
}

```

Médio: acessos constantes mais $O(k)$ na lista ou $O(h)$ na árvore; pior caso $O(n)$ se lista/árvore degenerarem.

Gabarito dos simulados

Simulado 1

1. Espera-se somatório dependente do laço interno, fórmula fechada e indução correta.
2. A correção deve mostrar todos os estados; as três estruturas devem preservar suas invariantes.
3. Cada afirmação precisa de contraexemplo ou argumento formal, não apenas V/F.
4. O comparador deve primeiro separar múltiplos de 3 e depois ordenar dentro dos grupos.
5. A resposta deve apresentar funções distintas para melhor e pior caso e entradas que as realizam.
6. A pesquisa deve seguir exatamente as camadas e somar os custos de cada uma.

Simulado 2

1. Bubble com ultimaTroca; pior caso $\Sigma(n-i)$ e $\Theta(n^2)$.
2. Criar nova árvore e podar ramos fora do intervalo; não compartilhar nós.
3. Caminhar inf e esq a partir do canto superior direito e copiar as listas.
4. Inserção e remoção devem manter cabeça e ultimo consistentes.
5. Desenho, fatores e código devem usar nível nulo 0.

6. Justificativas devem empregar melhor/pior caso e invariantes das estruturas.

Simulado 3

1. Solução $O(n)$ deve retornar validade, limites e nível em uma única travessia.
2. A representação alvinegra deve ser equivalente à 2-3-4.
3. Localizar prefixo na ABB de filhos e explorar somente a subárvore relevante.
4. Remoção principal deve promover da reserva um elemento com a mesma posição hash.
5. Percorrer a primeira árvore com poda pelo intervalo e somar contagens das segundas árvores.
6. Complexidade deve aparecer como soma dos custos: $O(\log n + 1 + k + |palavra|)$ no caso balanceado/médio.